

# RTSP Livestream Overlay Web Application – Assignment

Build a simple web application that plays a livestream video from an RTSP source and allows users to create, manage, and display custom overlays on top of the video in real time.

This assignment is intended to evaluate your ability to work with video streaming, frontend interaction, backend APIs, and data persistence.

## Assignment Requirements

### 1. Landing Page

- The landing page should display a livestream video sourced from an RTSP URL.
- You may use tools such as RTSP.me or any RTSP-compatible streaming service to generate a temporary RTSP stream from an existing video.
- The video player should be embedded directly on the page.
- A clear Play option must be available to start the livestream.

### 2. Application Features

#### *Livestream Playback*

The application must support:

- Playing a livestream from a user-provided RTSP URL
- Basic playback controls: Play, Pause, Volume control
- RTSP compatibility is mandatory (directly or via an appropriate conversion mechanism)

#### *Overlay Functionality*

- Users should be able to add and manage overlays on top of the livestream video.
- Supported overlay types: Text overlays, Image or logo overlays (via image URL).
- Overlays must be freely movable (drag-and-drop positioning).
- Overlays must be resizable.
- Overlays must be visible and updated in real time on the livestream view.

### 3. CRUD APIs for Overlays & Settings

Implement backend APIs to manage overlay configurations. Each overlay should support the following properties:

- Position (x, y coordinates)
- Size (width, height or scale)

- Content (text or image URL)
- Overlay type (text/image)

CRUD operations required:

- Create
- Read
- Update
- Delete

## Required Technology Stack

- Backend: Python (Flask preferred)
- Database: MongoDB
- Frontend: React
- Video Streaming: Must support RTSP-compatible livestream playback

## Deliverables

### ***1. Code Repository***

- Backend (Flask + MongoDB)
- Frontend (React)
- Clear and structured project layout

### ***2. Documentation***

- README with setup instructions for frontend and backend
- Instructions to run the application locally
- How to provide or change the RTSP URL
- API documentation covering all CRUD endpoints with examples
- User guide explaining livestream playback and overlay management

### ***3. Demo Video (Mandatory)***

- Short screen-recorded video with voice-over explanation
- Starting the application
- Playing the RTSP livestream
- Creating, updating, and deleting overlays
- Demonstrating real-time overlay behavior

## Submission Instructions

- Share the GitHub repository link
- Attach or link the demo video recording
- Ensure all documentation is included in the repository or as separate files

### ***Submission Email Details***

All completed assignments, including the GitHub repository link and demo video, must be sent via email with the following details:

- To: rakesh@gonote.ai
- CC: safiya@gonote.ai, aman@gonote.ai